

Q. ① Reg I: store Data from common Bus

② Dec B: Decode Data from Reg I and used to select appropriate control circuit

③ Status Reg.: store the results of certain condition when certain operation are performed during execution of the program

④ HLDA: indicates that the 8086 has entered the hold state.

⑤ M/I/O: indicates that 8086 address Bus contains either memory address or an I/O port address

⑥ Pipelining: BIU and EU operate asynchronously to give the 8086 an overlapping instruction fetch and execution mechanism

CLK: Provides the basic timing for processor operation and bus activity

$$D_{is} = \sigma A - B \quad (V)$$

$$D_{is \%} = \frac{D_{is}(V)}{A(V)} \times 100$$

$$A = \sqrt{P_{out}(W)} \cdot (V)$$

$$\sigma = X_{A-B} \times T_P$$

$$T_P = \frac{1}{R_F}$$

$$\sigma \% = X_{A-B} \times 100$$

$$S = \frac{\sigma_{A-B}}{X_{A-B} \times T_P} \quad (V/\mu s)$$

Perfect $\leq 0.1\%$

Excellent 0.1%

Very good 3%

Good 5%

Accepted 7%

Bad 8%

Very bad 10%

Not Accepted $> 10\%$

↑
Pipelining: reduce the time required for execution of the instruction

① 1-state : Pass a 1, a 0 or it can essentially disconnect its output

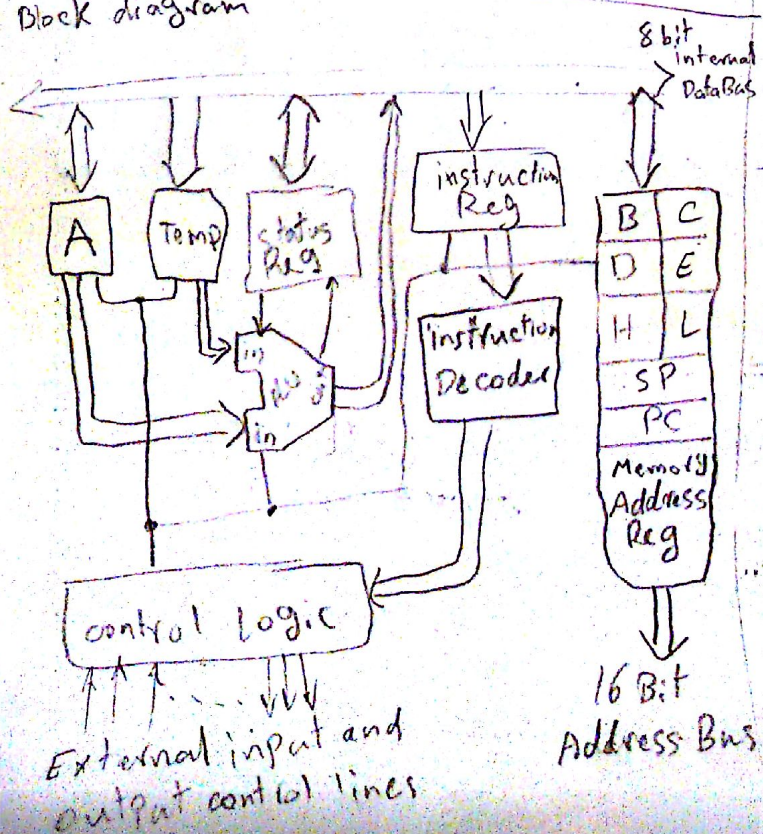
Q General Purpose Reg!

They are used as simple storage area, and to store intermediate results of operation.

Q. Flag Reg. - indicate the status of the CPU or the result of arithmetic operations.

	0	1
Control		
Direction (D)	up	down
Interrupt (IF)	disable	enable
* Trap (T)	off	the programmer can single step through his program to debug any errors
* Carry (C)	no carry	Carry
* overflow (O)	no overflow	overflow
* Sign (S)	Positive	Negative
* Zero (Z)	The result is not 0	the result is 0
* Auxiliary carry (A)	No carry	Carry
* Parity (P)	# of 1's is odd	# of 1's is even
MN/MX	maximum mode	minimum mode
ALE	data bits	address bits
Address Latch Enable		
READY	wait states	no effect
Test	wait instruction functions as NOP	Wait instruction wait for test become 0
DT/R	receiving Data	Transmitting Data
HOLD	executes software normally	Store executing Places address Data, control Bus at high impedance state

Block diagram



V_{CC}, V_{SS}: Power supply pins
V_{CC} → +5V V_{SS} → ground I = 125 mA

XTAL2, XTAL1: Oscillator pins

ALE: AD₀ to AD₇ lines are multiplexed
To demultiplex these lines and for obtaining
lower half of an address, an ALE signal is used

Rst: This pin is used to Reset 8051

PSEN: it's used to activate the enable
signal of the external Rom/EPROM

Port 0: P₀ (P_{0.0} ~ P_{0.7})

- * 8bit R/W - General Purpose I/O
- * Low byte address and Data byte bus for external memory

Port 1: P₁ (P_{1.0} ~ P_{1.7})

- * only 8bit R/W - General Purpose I/O

Port 2: P₂ (P_{2.0} ~ P_{2.7})

- * 8bit R/W - General purpose I/O
- * high byte address for external memory

Port 3: P₃ (P_{3.0} ~ P_{3.7})

- * General Purpose I/O
- * Timer (T₀, T₁)
- * RD, WR
- * ext. int (INT₀, INT₁)
- * serial (TXD, RXD)

Each port can be used as input or output
(bi-direction)

int. RAM organized into 3 distinct areas:

- * Working registers 128 byte
- * Bit addressable
- * General Purpose

ROM: 4 Kbyte

source of interrupts

- * Timer 1 overflow
- * Timer 2 overflow
- * External interrupt 0
- * External interrupt 1
- * serial port events

copy byte in SCon to Reg 3:

M1: using direct address for SCon (98H)

MOV R3, 98H

M2: using direct address for SCon and R3
(03H)

MOV 03H, 98H

M3: using indirect address for R3

MOV R1, #03H

MOV @R1, 98H

Adding 16-bit Num:

MOV DPTR, #2040H

MOV A, #2BH

MOV B, #20H

ADD A, DPL

MOV DPL, A

MOV A, B

ADDC A, DPH

MOV DPH, A

2's complement:

MOV A, R0

CPL A

ADD A, #01

Generate BCD up counter:

MOV A, #00

Back: MOV P1, A

ADD A, #01

DA A

AJMP Back

Unpack BCD:

```
MOV B, A
ANL A, #0Fh
MOV R0, A
MOV A, B
ANL A, #0F0h
SWAP A
MOV R1, A
```

subtract

```
MOV PSW, #10h
MOV A, R0
MOV PSW, #00h
CLR C
SUBB A, R1
```

Division two 8-bit Num:

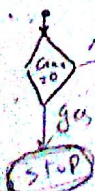
```
MOV A, #90
MOV B, #20
DIV AB
```

Multiply two 8-bit num:

```
MOV A, #BF
MOV B, #79
MUL AB
```

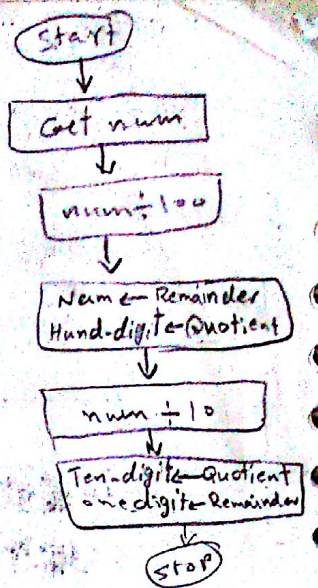
Max num of 10:

```
MOV DPTR, #2000
MOV R0, #0A
MOV R3, #00
AGAIN: MOVX A, @DPTR
CJNZ A, R3, NE
AJMP SKIP
NE: JC SKIP
MOV R3, A
SKIP: INC DPTR
DJNZ R0, AGAIN
```



Binary to BCD

```
MOV A, #16H
MOV B, #100
DIV AB
MOV R0, A
MOV A, B
MOV B, #10
DIV AB
MOV R1, A
MOV R3, B
```



Number of 1's

```
MOV R2, #0
MOV R1, #08
MOV R0, #56
MOV A, R0
Back: RRC
      JNC SKIP
      INC R2
SKIP: DJNZ R1, Back
```

count positive and Negative

```
MOV R0, #100
MOV R1, R0
MOV R2, #10AH
MOV DPTR, #2000H
Back: MOVX A, @DPTR
      ANL A, #80H
      JZ SKIP
      INC R0
      AJMP LAST
SKIP: INC R1
LAST: INC DPTR
      DJNZ R2, Back
```

